

Limitations of Linear Models

Linear boundary *Def:* a linear classifier (LogReg, perceptron, softmax pairwise) predicts by the sign of $z = \mathbf{w}^T \mathbf{x} + b$ ($\sigma(z) \geq 0.5 \iff z \geq 0$) $\rightarrow$ boundary is the hyperplane $\{\mathbf{x} : \mathbf{w}^T \mathbf{x} + b = 0\}$.

Linearly separable *Def:* two sets of points are linearly separable if there exists a hyperplane that perfectly separates them. *Effect if not:* no linear classifier reaches 100% training accuracy, no matter how much data or which optimizer.

Half-space *Def:* $H = \{\mathbf{x} \in \mathbb{R}^d : \mathbf{c}^T \mathbf{x} + b \geq 0\}$ ($\mathbf{c} \in \mathbb{R}^d, b \in \mathbb{R}$) = all points on one side of a hyperplane; $\{\mathbf{c}^T \mathbf{x} + b = 0\}$ is the hyperplane itself.

Convex set *Def:* $S \subseteq \mathbb{R}^d$ is convex if $\forall \mathbf{x}^{(a)}, \mathbf{x}^{(b)} \in S, \forall \lambda \in [0, 1]: \lambda \mathbf{x}^{(a)} + (1 - \lambda) \mathbf{x}^{(b)} \in S$ (the segment between any 2 points of S stays in S).

Half-space is convex *Proof:* $\mathbf{x}^{(a)}, \mathbf{x}^{(b)} \in H, \lambda \in [0, 1]$, split $b = \lambda b + (1 - \lambda)b$:

$$\begin{aligned} \mathbf{c}^T (\lambda \mathbf{x}^{(a)} + (1 - \lambda) \mathbf{x}^{(b)}) + b &= \lambda (\mathbf{c}^T \mathbf{x}^{(a)} + b) + (1 - \lambda) (\mathbf{c}^T \mathbf{x}^{(b)} + b) \geq 0 \\ &= \lambda (\mathbf{c}^T \mathbf{x}^{(a)} + b) + (1 - \lambda) (\mathbf{c}^T \mathbf{x}^{(b)} + b) \geq 0 \end{aligned}$$

XOR: $t = 1$ iff $x_1 \neq x_2$: $(0, 0) \rightarrow 0, (0, 1) \rightarrow 1, (1, 0) \rightarrow 1, (1, 1) \rightarrow 0$. *Theorem:* not linearly separable.

Algebraic proof (XOR): *Constraints:* $t = 1 \Rightarrow b + w_1 x_1 + w_2 x_2 \geq 0$; $t = 0 \Rightarrow b + w_1 x_1 + w_2 x_2 < 0$; $(0, 0)$: $b < 0$; $(0, 1)$: $b + w_2 \geq 0$; $(1, 0)$: $b + w_1 \geq 0$; $(1, 1)$: $b + w_1 + w_2 < 0$.

Contradiction: add the 2 positive constraints: $2b + w_1 + w_2 \geq 0$; Add the 2 negative ones: $2b + w_1 + w_2 < 0$.

Geometric proof *Steps:* (any dataset, by contradiction) (1) Find 2 class-1 points A, C and 2 class-0 points B, D whose segments $\overline{AC}, \overline{BD}$ intersect.

(2) Intersection F : solve $\lambda A + (1 - \lambda)C = \mu B + (1 - \mu)D$ (2 eqs, 2 unknowns), check $\lambda, \mu \in [0, 1]$. *Shortcut:* if $\frac{A+C}{2} = \frac{B+D}{2}$ then F = that midpoint.

(3) Assume separable: $\exists f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ with class 1 in $H^+ = \{\mathbf{x} : f(\mathbf{x}) \geq 0\}$, class 0 in $H^- = \{\mathbf{x} : f(\mathbf{x}) < 0\}$.

(4) H^+, H^- are half-spaces $\Rightarrow$ convex.

(5) $A, C \in H^+$ and H^+ convex $\Rightarrow \overline{AC} \subseteq H^+ \Rightarrow F \in H^+$.

(6) $B, D \in H^-$ and H^- convex $\Rightarrow F \in H^-$.

(7) $H^+ \cap H^- = \emptyset, F$ can't be in both $\Rightarrow$ contradiction $\Rightarrow$ not linearly separable. $\square$

Why (5), algebraically: f affine $\Rightarrow f(F) = \lambda f(A) + (1 - \lambda) f(C) \geq 0$, but also $f(F) = \mu f(B) + (1 - \mu) f(D) < 0$. (Algebraic proof = this at the midpoint F .)

Variant: a class-0 point P lies on a class-1 segment ($P = \lambda A + (1 - \lambda)C$, 3 collinear points) or inside a class-1 triangle $\Rightarrow f(P) \geq 0$, contradicts $P \in H^-$.

Intuition: separable $\iff$ the convex hulls of the 2 classes don't intersect.

Feature map *Fix:* add $x_3 = \phi(\mathbf{x})$ (e.g. $x_1 x_2, x_2^2$) so the classes become separable in the extended space; Boundary is linear in (x_1, x_2, x_3) , nonlinear after substituting $x_3 = \phi(\mathbf{x})$ back.

Steps: pick ϕ on which the classes differ $\rightarrow$ add the x_3 column to the table $\rightarrow$ pick $(b, \mathbf{w})$ satisfying every row's constraint $\rightarrow$ substitute $x_3 = \phi(\mathbf{x})$ for the original-space boundary.

XOR: $x_3 = x_1 x_2, (b, w_1, w_2) = (-0.5, 1, 1, -2) \rightarrow x_1 + x_2 - 2x_1 x_2 - 0.5 = 0 \iff (2x_1 - 1)(2x_2 - 1) = 0 \rightarrow$ lines $x_1 = 0.5, x_2 = 0.5$.

Cons: needs domain knowledge + manual experimentation; #features grows combinatorially (d inputs $\rightarrow O(d^2)$ degree-2 products); Overfits without regularization.

Motivation for NN: learn the features from data instead of hand-crafting them; Last layer of a NN = linear classifier on learned features, $\mathbf{w}^T \phi(\mathbf{x}) + b$.

Introduction to Neural Networks

2.1 The Neuron

Artificial neuron *Def:* $y = f(\mathbf{w}^T \mathbf{x} + b)$; $\mathbf{x}$ = inputs (features), $\mathbf{w}$ = weights (learned), b = bias, $z = \mathbf{w}^T \mathbf{x} + b$ = pre-activation, f = activation function (usually nonlinear), y = output (activation).

Biology map: dendrites = input channels; Synapse = weight (strength & sign, negative = inhibitory); Cell body = $\sum_j w_j x_j + b$ (integrates); Bias = intrinsic excitability; f = whether/how strongly it fires; Axon = output channel. **Output:** real-valued (like a firing rate), not binary (unless f = threshold).

Linear models as NN: f = identity $\rightarrow$ LinReg; $f = \sigma \rightarrow$ LogReg = a single neuron; Softmax regression = 1-layer NN with K output units, $\mathbf{y} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$. *Cons:* 1 affine map + activation $\rightarrow$ only linear boundaries.

Network diagram: node = a value, arrow = weighted connection; Computation (multiply, sum, bias, f) is implied, not drawn. *Conventions:* vertical/horizontal layout, inputs as nodes, bias as a node (= weight from a constant 1): same network, only the drawing changes. **Computation graph:** every numerical step explicit (used for back-prop).

2.2 MLP Definitions

Unit *Def:* any node in a NN. **Neuron** *Def:* a unit that computes its output from its inputs. *Contrast:* input units only hold data (no computation) $\rightarrow$ units but not neurons. **Input layer** *Def:* represents the input features; Does no computation.

Computational layer *Def:* a layer whose units are all neurons; Hidden + output layers are computational, the input layer is not.

Output layer *Def:* computational layer producing values with a defined interpretation (class probabilities / continuous value).

Hidden layer *Def:* any computational layer between the input and output layers; Its outputs = internal representations (learned features, not hand-designed; neither observed data nor final prediction).

Feedforward NN *Def:* computation flows only from ear-

lier to later layers, no feedback loops (a DAG). **Recurrent NN** *Def:* has cycles (not in course).

Fully connected layer *Def:* each unit receives input from every unit of the previous layer, and only from those units (no skip edges).

MLP *Def:* feedforward NN with ≥ 1 hidden layers between input and output, every layer fully connected.

Perceptron *Def:* Rosenblatt's (1958) neuron, weighted sum + hard threshold; "MLP" name is historical (modern nets use smooth f).

Inference *Def:* computing the network's output for a given input. **Forward pass** *Def:* the computation doing it (input $\rightarrow$ output). *Training:* adds the loss $\mathcal{L}(\mathbf{a}^{(M)}, \mathbf{t})$ (needs known $\mathbf{t}$); Forward pass + loss = phase 1 of back-prop; All steps differentiable $\rightarrow$ trainable by GD.

Roles: hidden layers build intermediate features; Output layer turns them into a task-meaningful prediction.

2.3 Counting Layers

Rule: $M = \# \text{layers} = \# \text{computational layers} = \# \text{hidden} + 1$; Never count the input layer.

#Neurons: hidden + output units only; **#Units:** also counts the input units.

Layer of a neuron (general DAG): input units = layer 0; Neuron's layer = $1 + \max(\text{layers of the units feeding it})$ (longest path from the inputs).

Naming: LogReg / softmax = 1-layer NN (no hidden); "2-layer NN" = 1 hidden + output.

Ex 1: $4 \rightarrow 5 \rightarrow 3 \rightarrow 1 = 3$ -layer MLP ($M = 3$, 2 hidden layers, 9 neurons, 13 units).

Ex 2: inputs A, B ; edges $A \rightarrow C, B \rightarrow C, A \rightarrow D, C \rightarrow D, D \rightarrow E$ $\rightarrow$ neurons C, D, E ; Layers $C = 1, D = 2, E = 3 \rightarrow 3$ layers; Feedforward (no loops); $A \rightarrow D$ skips layer 1 $\rightarrow$ not fully connected $\rightarrow$ not an MLP.

#Params: $\sum_{m=1}^M D^{(m)}(D^{(m-1)} + 1)$ (weights $D^{(m)}D^{(m-1)} + \text{biases } D^{(m)}$). *Ex 1:* $5(4+1) + 3(5+1) + 1(3+1) = 47$.

2.4 Notation & Forward Pass

Symbol	Dim	Meaning
$\bar{M}$	scalar	#computational layers
m	$\{0, \dots, M\}$	layer index: 0 input, 1..M-1 hidden, M output
$D^{(m)}$	scalar	#units in layer m
$\mathbf{a}^{(0)} = \mathbf{x}$	$D^{(0)} \times 1$	network input
$\mathbf{W}^{(m)}$	$D^{(m)} \times D^{(m-1)}$	weights into layer m (out $\times$ in)
$\mathbf{b}^{(m)}$	$D^{(m)} \times 1$	biases of layer m
$f^{(m)}$	$\mathbb{R} \rightarrow \mathbb{R}$	activation, element-wise (softmax: whole vector)
$\mathbf{z}^{(m)}$	$D^{(m)} \times 1$	pre-activations of layer m
$\mathbf{a}^{(m)}$	$D^{(m)} \times 1$	activations of layer m
$\mathbf{h}^{(m)} = \mathbf{a}^{(m)}$	$D^{(m)} \times 1$	hidden activations, $m \in \{1, \dots, M-1\}$
$\mathbf{y} = \mathbf{a}^{(M)}$	$D^{(M)} \times 1$	network output
$\mathbf{t}$	$D^{(M)} \times 1$	target
$\mathcal{L}(\mathbf{a}^{(M)}, \mathbf{t})$	scalar	loss for one example

Weight index: $W_{j,k}^{(m)}$ = weight to unit j of layer m from unit k of layer $m-1$ (1st index = receiver). *Why $\mathbf{W}\mathbf{a}$:* $\mathbf{W}^{(m)}$ is $(\text{out} \times \text{in}) \rightarrow \mathbf{W}^{(m)} \mathbf{a}^{(m-1)} \in \mathbb{R}^{D^{(m)}}$.

Forward pass ($m = 1, \dots, M$): *Scalar:*

$$z_j^{(m)} = \sum_{k=1}^{D^{(m-1)}} W_{j,k}^{(m)} a_k^{(m-1)} + b_j^{(m)}, \quad a_j^{(m)} = f^{(m)}(z_j^{(m)})$$

Vectorized: $\mathbf{z}^{(m)} = \mathbf{W}^{(m)} \mathbf{a}^{(m-1)} + \mathbf{b}^{(m)}, \quad \mathbf{a}^{(m)} = f^{(m)}(\mathbf{z}^{(m)})$

Loss: $\mathcal{L} = \mathcal{L}(\mathbf{a}^{(M)}, \mathbf{t})$; *Cost* $\mathcal{E}$ = average of $\mathcal{L}$ over examples.

Output layer *Why only vectorized:* softmax is not element-wise: each y_j depends on the whole $\mathbf{z}^{(M)}$. *Reg, 1 output:* $f^{(M)} = \text{identity}, y = z_1^{(M)}$.

Inference *Steps:* $\mathbf{z}^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \rightarrow \mathbf{a}^{(1)} = f^{(1)}(\mathbf{z}^{(1)})$ (threshold: $z \geq 0 \rightarrow 1$) $\rightarrow$ repeat per layer $\rightarrow$ softmax output $y_k = e^{z_k} / \sum_k e^{z_k} \rightarrow$ predict $\arg \max_k y_k$; Label which values are input / hidden / output.

Units affected by $\mathbf{W}_{j,k}^{(m)}$: only $z_j^{(m)}, a_j^{(m)}$ in layer m (other units of layer m and all earlier layers unchanged), then every unit of layers $m+1, \dots, M$ (fully connected), then $\mathcal{L}$. *Equations to write:* exactly those z 's, a 's and $\mathcal{L}$, in forward order.

2.5 Designing an MLP

Decision	How chosen
#input units	data representation (D features; 28 $\times$ 28 image flattened $\rightarrow \mathbf{x} \in \mathbb{R}^{784}$, 1 unit per pixel)
Target	one-hot $\mathbf{t} \in \mathbb{R}^K$; class index from 0 $\rightarrow$ digit k has 1 at entry $k+1$
#hidden layers	hyperparameter (tune on val); #comp. layers = this + 1
#units per hidden layer	hyperparameter
Hidden activation	hyperparameter (default ReLU)
#output units	task: Reg 1; Binary 1; K-class K
Output activation	task: Reg identity; Binary σ ; K-class softmax

Task	$D^{(M)}$	$f^{(M)}$	Target	Loss
Reg	1	identity	$\mathbf{t} \in \mathbb{R}$	SE $\frac{1}{2}(y - t)^2$
Binary	1	σ	$\mathbf{t} \in \{0, 1\}$	CE
K-class	K	softmax	one-hot $\mathbf{t} \in \{0, 1\}^K$	CE $-\mathbf{t}^T \log \mathbf{y}$

Softmax derivative: $\frac{\partial y_i}{\partial z_j} = y_i(\mathbb{I}[i = j] - y_j)$ ($i = j$: $y_i(1 - y_i)$; $i \neq j$: $-y_i y_j$). *With CE:* $\frac{\partial \mathcal{E}}{\partial z_i} = \mathbf{y} - \mathbf{t}$.

2.6 Activation Functions

Activation function *Def:* $f : \mathbb{R} \rightarrow \mathbb{R}$, typically a nonlinear scalar function, applied element-wise to $\mathbf{z}^{(m)}$: $\mathbf{a}^{(m)} = f(\mathbf{z}^{(m)})$. *Exception:* softmax (output layer, K-class) acts on the whole vector, still called an activation function.

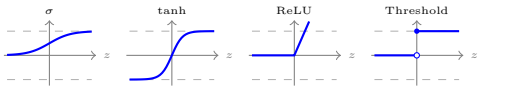
Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$; $\sigma' = \sigma(1 - \sigma) \leq 0.25$ (max at $z = 0$).

Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1 \in (-1, 1)$; $\tanh' = 1 - \tanh^2 \leq 1$ (max at $z = 0$). *Shape:* sigmoid compressed $\times 2$ horizontally, stretched $\times 2$ vertically, shifted down 1 $\rightarrow$ passes through origin, odd (symmetric about origin).

ReLU: $\max(0, z) \in [0, \infty)$; $\text{ReLU}' = \mathbb{I}[z > 0]$ (not differentiable at $z = 0$; define 0 or 1, doesn't affect training). *Nonlinearity:* only the kink at $z = 0$, but enough for expressive power.

Threshold: $\mathbb{I}[z \geq 0] \in \{0, 1\}$ ($f(0) = 1$); Derivative 0 for $z \neq 0$, undefined at 0.

Graphs: dashed lines at ± 1 ; σ saturates at 0 / 1, tanh at $-1 / 1$; Threshold: open circle (0, 0), filled (0, 1).



Why nonlinear f : without it stacked linear layers collapse to 1 linear map; f is the only source of nonlinearity. **Saturated** *Def:* f flat when z far from 0 $\rightarrow$ a small change in incoming weights barely changes the output.

Dead neuron *Def:* GD stops updating a neuron's incoming weights because $f' \approx 0$ at the inputs it receives (even if its output is wrong; any depth).

Vanishing gradient *Def:* the deeper the net, the smaller the gradients for early-layer weights. *Cause:* chain rule gives one f' factor per layer between the weight and the output; Factors $< 1 \rightarrow$ product shrinks exponentially. *Effect:* early layers learn slowly / not at all.

Zero-centred *Def:* activations can be positive or negative (tanh).

Dying ReLU *Def:* $z < 0$ for every training input $\rightarrow$ always outputs 0, incoming weights never update. *Fix:* Leaky ReLU / ELU (small nonzero output for $z < 0$).

Sigmoid all-positive: *Effect:* next layer's inputs all $> 0 \rightarrow$ each update increases all or decreases all of a unit's incoming weights $\rightarrow$ zigzag, slow. *Fix:* tanh (zero-centred).

	GD	Dead neuron	Vanishing grad
Sigmoid	Yes: diff. $\forall z$	No: $f' \neq 0$	Yes: $f' \rightarrow 0$ for $ z $ large
Tanh	Yes: diff. $\forall z$	No: $f' \neq 0$	Yes: $f' \rightarrow 0$ for $ z $ large
ReLU	Yes: diff. except $z = 0$	Yes: $f' = 0$	No: $f' = 1$ for $z > 0$
Threshold	No: $f' = 0$ a.e.	Yes: $f' = 0$	Yes: $f' = 0$ a.e.

Poor-gradient inputs: $\sigma, \tanh$: $|z|$ large; ReLU: $z < 0$. **Sigmoid Pros:** smooth, everywhere-differentiable approximation of threshold ($\rightarrow$ GD works); Output = firing probability.

ReLU Pros: cheap (one comparison); Near-linear (identity for $z > 0$) $\rightarrow$ easy to optimize; Slope 1 $\rightarrow$ no vanishing gradient $\rightarrow$ enables deep nets.

Choice (hyperparameter, hidden & output chosen separately; judged by how well GD trains $\rightarrow$ Idea #1 Learning is optimization): *Hidden:* must be nonlinear + support GD; ReLU default; Tanh if zero-centred matters; Sigmoid rarely. *Output:* by task (identity / σ / softmax: interpretable output matching the loss). *Threshold:* only for perceptron history & hand-picked weights.

3 Expressiveness of Neural Networks

Linear activations collapse: $f = \text{identity} \Rightarrow \mathbf{y} = \mathbf{W}^{(2)}(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} = \mathbf{W}_{eq} \mathbf{x} + \mathbf{b}_{eq}$,

$$\mathbf{W}_{eq} = \mathbf{W}^{(2)} \mathbf{W}^{(1)}, \quad \mathbf{b}_{eq} = \mathbf{W}^{(2)} \mathbf{b}^{(1)} + \mathbf{b}^{(2)}$$

Affine $f(z) = az + c$ (per neuron, $\mathbf{a} = \text{diag}(\mathbf{a}), \mathbf{c}$ stacked): $\mathbf{W}_{eq} = \mathbf{W}^{(2)} \mathbf{A} \mathbf{W}^{(1)}, \mathbf{b}_{eq} = \mathbf{W}^{(2)} (\mathbf{A} \mathbf{b}^{(1)} + \mathbf{c}) + \mathbf{b}^{(2)} \rightarrow$ still linear. *Effect:* any depth of linear layers = 1 linear layer ($= \text{LinReg}$).

Threshold unit ($f(z) = \mathbb{I}[z \geq 0], x_j \in \{0, 1\}$) *Templates:* AND $f(x_1 + x_2 - 1.5)$; OR $f(x_1 + x_2 - 0.5)$; NOT $f(-x_1 + 0.5)$; NAND $f(-x_1 - x_2 + 1.5)$; $x_1 \wedge x_2 = f(x_1 - x_2 - 0.5)$.

How $\mathbf{w}, b$ decide: $w_j > 0$: $x_j = 1$ pushes toward 1; $w_j < 0$: needs $x_j = 0$; b = threshold (how many conditions must hold) $\rightarrow$ $\mathbf{w}$ = normal of the boundary, points to the class-1 side.

Steps to hand-pick: truth table $\rightarrow$ plot the 4 points $\rightarrow$ draw a separating line $\rightarrow$ write it as $\mathbf{w}^T \mathbf{x} + b = 0$, sign chosen so the class-1 side is $\geq 0 \rightarrow$ verify every row.

XOR, 2-layer: (a) $x_1 \oplus x_2 = (x_1 \wedge x_2) \vee (\neg x_1 \wedge x_2)$: $h_1 = f(x_1 - x_2 - 0.5), h_2 = f(-x_1 + x_2 - 0.5), y = f(h_1 + h_2 - 0.5)$. (b) $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$: $h_1 = f(x_1 + x_2 - 0.5)$ (OR), $h_2 = f(-x_1 - x_2 + 1.5)$ (NAND), $y = f(h_1 + h_2 - 1.5)$ (AND).

Row detector: unit fires iff $\mathbf{x} = \mathbf{x}^{(i)}$: $w_j = +1$ if $x_j^{(i)} = 1, -1$ if $x_j^{(i)} = 0$; $b = -(\#1\text{s in } \mathbf{x}^{(i)}) + 0.5$. *Why:* $\mathbf{w}^T \mathbf{x} \leq \#1\text{s}$, equality only at $\mathbf{x}^{(i)}$.

Any Boolean function, 1 hidden layer: one row detector per $t = 1$ row + OR output ($w = 1$ each, $b = -0.5$). *Cons:* may need up to 2^D hidden units (exponential). *Ex:* 3-bit parity: rows 001, 010, 100, 111 $\rightarrow 4$ hidden units.

Universal approximation: 1 hidden layer with nonlinear f and enough units can approximate any continuous function (on a bounded domain) arbitrarily well. *Cons:* may need exponentially many units.

4 The Backpropagation Algorithm

5 Bias-Variance

Decomposition, Ensembles

6 Statistical Learning

7 Naive Bayes

8 Gaussian Discriminant Analysis